

FAZAIA BILQUIS COLLEGE OF EDUCATION FOR WOMEN, PAF BASE NUR KHAN



SEMESTER PROJECT

Computer Organization and Assembly Language (COAL)

Assembly Language Quiz Management System with Web-Based Analytics Dashboard

Submitted By:	Aiman Munawar Aimen Khan
Roll Numbers:	BSCS-13F24-14 BSCS13-F24-24
Semester:	Spring 2026
Submitted To:	Ms. Sana
Submission Date:	May 2026

Developed using 8086 Assembly Language (TASM/DOSBox) & Web Technologies

Abstract

This report presents the design and implementation of an Assembly Language Quiz Management System integrated with a Web-Based Analytics Dashboard. The project is developed as part of the Computer Organization and Assembly Language (COAL) course and serves as a practical demonstration of low-level programming concepts combined with modern front-end web development.

The core quiz engine is implemented using 8086 Assembly Language, assembled and executed through TASM (Turbo Assembler) and DOSBox. It allows users to register their usernames, select from multiple quiz categories including General Knowledge, Science, History, Sports, and Movies, and participate in an interactive multiple-choice quiz. Scores are computed using Assembly-level arithmetic operations, and results are persistently stored in a plain-text file (RECORD.TXT) using DOS interrupt-based file handling (INT 21h).

The web-based front-end provides an intuitive graphical interface offering five primary navigation options: Home, Start Quiz, View Records, Dashboard Analytics, and About Project. Users upload the RECORD.TXT file generated by the Assembly program, which is then parsed and analyzed to generate statistical visualizations including pie charts for score distribution, bar charts for category performance, summary statistics, and a recent activity log.

The system demonstrates a novel integration between classical low-level assembly programming and contemporary web technologies, making it educationally significant for Computer Science students studying both system-level programming and software development.

Table of Contents

1. Introduction	5
2. Problem Statement	5
3. Objectives	6
4. Scope of Project	6
5. Literature Review	6
6. Technologies Used	7
7. System Analysis	7
8. Proposed System	8
9. System Architecture	8
10. Flowchart	9
11. Algorithm	9
12. Assembly Language Implementation	10
13. File Handling in Assembly	10
14. Frontend Design	11
15. Dashboard Analytics	11
16. Record Processing Mechanism	12
17. Screens Description	12
18. Advantages	13
19. Limitations	13
20. Future Enhancements	13
21. Testing and Results	14
22. Conclusion	14
23. References	15

1. Introduction

The study of Computer Organization and Assembly Language forms the backbone of computer science education, providing students with a fundamental understanding of how hardware and software interact at the lowest level. Assembly language programming, despite being considered a legacy skill by many practitioners, remains critically important for understanding processor architecture, memory management, and system-level programming.

This project, titled "Assembly Language Quiz Management System with Web-Based Analytics Dashboard," bridges the gap between classical low-level programming and modern web development. It comprises two integrated components: a quiz engine written in 8086 Assembly Language executed through DOSBox, and a responsive web application developed using HTML5, CSS3, and JavaScript that reads the quiz results and renders them as interactive visual analytics.

The project is structured to challenge students in both traditional and modern programming paradigms simultaneously. The Assembly component requires mastery of registers, memory segments, DOS interrupts, and file I/O, while the web component demands proficiency in DOM manipulation, data parsing, and data visualization using charting libraries.

2. Problem Statement

Educational institutions routinely conduct assessments to evaluate student performance. However, in the context of Assembly Language courses, there are few practical examples that combine system-level programming with real-world application development. Students often learn Assembly Language in isolation, without understanding how it can interact with other technologies.

Moreover, quiz and assessment systems built with high-level languages abstract away the underlying data handling and storage mechanisms. There is a genuine need for a system that:

- Demonstrates real-world applicability of Assembly Language programming beyond theoretical exercises.
- Implements persistent data storage using low-level DOS file handling without reliance on any runtime or framework.
- Provides meaningful performance analytics to educators and students through a web-based visual interface.
- Serves as an educational model showing interoperability between low-level and high-level systems.

This project addresses these needs by implementing a complete quiz lifecycle from question delivery to performance analytics.

3. Objectives

The primary objectives of this project are as follows:

1. To design and implement a fully functional quiz engine using 8086 Assembly Language that supports multiple categories and computes scores accurately.
2. To implement file handling using DOS INT 21h interrupts for persistent storage of quiz records in RECORD.TXT.
3. To develop a user-friendly web interface that serves as the front-end for the quiz management system.
4. To create an analytics dashboard that parses the stored records and generates visual representations including pie charts and bar charts.
5. To compute and display key performance statistics such as total players, highest score, average score, and popular category.
6. To demonstrate integration between a legacy Assembly Language system and a modern web-based interface.
7. To document the complete system design, implementation, and testing in an academic report.

4. Scope of Project

The scope of this project encompasses the following dimensions:

4.1 Technical Scope

- Development of an 8086 Assembly Language quiz engine using TASM and DOSBox.
- Implementation of five quiz categories: General Knowledge, Science, History, Sports, and Movies.
- Each category contains multiple-choice questions evaluated at runtime.
- Results are written to a plaintext file using DOS interrupt services.
- A complete web application with five distinct pages providing navigation and analytics.

4.2 Educational Scope

- The project serves as a practical case study for COAL course students.
- It illustrates how Assembly Language programs can interface with external systems.
- The web dashboard demonstrates data analysis and visualization skills.

5. Literature Review

Assembly language programming has been extensively studied as a foundation of computer science. According to Patterson and Hennessy (2017), understanding processor-level instructions is essential for optimizing software performance and building system software such as compilers and operating systems.

DOS-based file handling using INT 21h services has been documented in depth by Irvine (2010) in "Assembly Language for x86 Processors," which describes the complete set of function calls available for creating, reading, writing, and closing files in DOS environments. These techniques remain pedagogically valuable despite the obsolescence of DOS in production environments.

The integration of legacy systems with modern web interfaces is a recurring theme in enterprise software. Studies in software interoperability (Sommerville, 2016) demonstrate that data exchange through structured plaintext files remains a robust and universally compatible integration strategy, even across significantly different technology generations.

Data visualization in web applications has been advanced significantly by JavaScript charting libraries. Chart.js, used in this project, is an open-source library described in its documentation as a simple yet flexible tool for designers and developers to create beautiful charts. Research by Few (2009) in "Now You See It" emphasizes that well-designed dashboards can significantly improve data comprehension and decision-making speed.

The pedagogical value of combining multiple programming paradigms within a single course project is supported by Bloom's Taxonomy (revised by Anderson et al., 2001), which highlights the importance of creating artifacts as the highest level of cognitive engagement in learning.

6. Technologies Used

Technology / Tool	Purpose
8086 Assembly Language	Core quiz engine development
TASM (Turbo Assembler)	Assembling .asm files to .obj and .exe
DOSBox	Emulation environment for running 16-bit DOS programs
INT 21h DOS Interrupts	File I/O, string output, character input
HTML5	Webpage structure and semantic markup

CSS3	Styling, responsive layout, dark theme design
JavaScript (ES6+)	Client-side logic, file parsing, dynamic rendering
Chart.js	Pie chart and bar chart visualizations
RECORD.TXT	Shared plaintext data store between subsystems

7. System Analysis

7.1 Existing System Analysis

Most existing quiz systems are developed entirely using high-level languages such as Python, Java, or PHP. These systems rely on databases, runtime libraries, and network connectivity. While they offer advanced features, they fail to demonstrate the underlying data management mechanisms at the system level. For a COAL course, such tools are didactically insufficient because they conceal the very operations — memory access, register manipulation, system calls — that the course aims to teach.

7.2 Proposed System Analysis

The proposed system adopts a two-tier architecture. The first tier is a standalone 8086 Assembly Language program that manages quiz execution locally without any runtime dependency other than a DOS emulator. The second tier is a web application that independently processes the output of the first tier. This architecture ensures a clean separation of concerns and highlights the role of standardized data formats as a bridge between heterogeneous systems.

7.3 Feasibility Analysis

- Technical Feasibility: Both TASM/DOSBox and web technologies are freely available and well-documented.
- Operational Feasibility: The system is usable without any server-side infrastructure, making it deployable in any academic computing lab.
- Economic Feasibility: All tools used are open-source or freeware, incurring zero licensing cost.
- Time Feasibility: The project scope was designed to be completable within a single academic semester.

8. Proposed System

The proposed system is a dual-component application that leverages the strengths of both Assembly Language and web technologies. The system workflow is as follows:

8. The user launches the web interface (index.html) in any modern browser.
9. The home page presents the system overview and navigation options.
10. The user proceeds to the "Start Quiz" page, which instructs them to run the COAL.EXE assembly program in DOSBox.
11. Inside the Assembly program, the user enters their username, selects a quiz category, and answers multiple-choice questions.
12. Upon quiz completion, the Assembly program appends the result to RECORD.TXT in the format: Username: [name] | Category: [cat] | Score: [n].
13. The user returns to the web interface and navigates to "View Records" or "Dashboard."
14. The user uploads the RECORD.TXT file via the web interface.
15. The JavaScript parser reads and tokenizes the file, then populates the analytics dashboard with charts and statistics.

9. System Architecture

The system architecture is divided into two primary subsystems that communicate through a shared plaintext data file:

9.1 Assembly Subsystem

- Input Layer: Username entry via keyboard using INT 21h / AH=0Ah buffered input.
- Processing Layer: Category selection using branching (CMP/JE instructions), question delivery, answer validation, and score accumulation using ADD/INC operations.
- Storage Layer: File creation and writing using INT 21h (AH=3Ch, 3Dh, 40h, 3Eh) for RECORD.TXT.

9.2 Web Subsystem

- Presentation Layer: Five HTML pages providing a cohesive user interface styled with a dark purple-blue theme.
- Logic Layer: JavaScript modules handling file reading (FileReader API), record parsing (string splitting and regex), and statistical computation.
- Visualization Layer: Chart.js library rendering pie and bar charts dynamically based on parsed data.

9.3 Integration Layer

The two subsystems are coupled through RECORD.TXT. This file acts as a neutral data exchange format, allowing the Assembly program and the JavaScript application to operate independently while sharing a common data representation.

10. Flowchart

10.1 Assembly Quiz Engine Flowchart

The flowchart of the Assembly quiz engine follows this logical sequence:

- START: Initialize data segment and stack segment.
- Display welcome message and prompt for username.
- Read and store username using buffered input (INT 21h / AH=0Ah).
- Display category menu and read selection.
- Branch to selected category subroutine.
- Display questions sequentially; read single-character answers (INT 21h / AH=01h).
- Compare answer with correct key; if match, increment score register.
- Repeat for all questions in category.
- Convert score to ASCII and display result message.
- Open/Create RECORD.TXT and write formatted record.
- Close file and display closing message.
- TERMINATE: INT 21h / AH=4Ch.

10.2 Web Dashboard Flowchart

- User opens web application in browser.
- Navigate to Dashboard page.
- Upload RECORD.TXT via file input.
- FileReader API reads file as text.
- Parser splits records by newline and tokenizes each line by pipe delimiters.
- Statistical engine computes totals, averages, and category distributions.
- Chart.js renders pie and bar charts.
- Statistics cards and Recent Activity table are populated.

11. Algorithm

11.1 Score Computation Algorithm (Assembly)

The score computation in Assembly follows this logical procedure:

16. Initialize score register: MOV BL, 0
17. For each question, display question text using INT 21h / AH=09h.
18. Read single character input: INT 21h / AH=01h → result in AL.
19. Compare answer: CMP AL, correct_answer_char.
20. If equal (JE), increment BL: INC BL.
21. After all N questions, convert BL to ASCII: ADD BL, 30h.
22. Store ASCII score for display and file writing.

11.2 Record Parsing Algorithm (JavaScript)

23. Read file content as string using FileReader.readAsText().
24. Split content by newline character to obtain individual records.
25. For each record, split by " | " pipe delimiter.
26. Extract Username (after "Username: "), Category (after "Category: "), and Score (after "Score: ").
27. Push parsed object { username, category, score } to records array.
28. Compute totalPlayers = records.length.
29. Compute highestScore = Math.max(...records.map(r => r.score)).
30. Compute averageScore = records.reduce((sum, r) => sum + r.score, 0) / totalPlayers.
31. Compute categoryDistribution by grouping records by category.
32. Compute scoreDistribution by classifying each score: 80-100=Excellent, 60-79=Good, 40-59=Average, 0-39=Needs Improvement.

12. Assembly Language Implementation

The Assembly Language program (COAL.ASM) is structured with the following key segments:

12.1 Data Segment

The data segment declares all string messages, question texts, answer buffers, and file name strings. Key declarations include:

- welcome_msg: DB 'Welcome to the Quiz System', '\$' — Display string terminated by dollar sign for INT 21h/AH=09h.
- username_buf: DB 20, 0, 20 DUP(0) — Buffered input structure for storing up to 20-character usernames.
- filename: DB 'RECORD.TXT', 0 — Null-terminated filename for file operations.

- category arrays and question strings for all five quiz categories.

12.2 Code Segment

The code segment contains the main procedure and multiple subroutines. Key techniques include:

- Conditional branching (CMP/JE/JNE) for menu navigation and answer validation.
- Loop constructs (LOOP instruction and CX register) for iterating through questions.
- Stack operations (PUSH/POP) for preserving register states across subroutine calls.
- String output using LEA DX, message followed by INT 21h with AH=09h.

13. File Handling in Assembly

File handling is implemented exclusively using DOS Interrupt 21h services, without any C runtime library or external dependencies. The following file operations are performed:

Operation	INT / AH	Description
Create/Open File	3Ch	Creates RECORD.TXT; if it exists, opens in write mode. CX=0 for normal attribute. Returns file handle in AX.
Open Existing File	3Dh	Opens file for reading or appending. AL=02h for read-write access. Returns handle in AX.
Move File Pointer	42h	Moves to end of file for appending. AL=02h (from end), CX:DX=0.
Write to File	40h	Writes CX bytes from buffer at DS:DX to file handle BX.
Close File	3Eh	Closes file handle in BX, flushing all pending writes.

The record is written in a human-readable pipe-delimited format to ensure compatibility with the JavaScript parser in the web interface.

14. Frontend Design

The web interface follows a consistent dark-themed design aesthetic with a deep navy and purple color palette, reflecting the academic and technical nature of the project. The interface consists of five pages:

14.1 Home Page

The home page serves as the landing screen. It displays the project title, a brief description, and navigation links. Animated accent elements and a gradient hero banner create a visually engaging first impression. Quick-access feature cards highlight the key capabilities of the system.

14.2 Start Quiz Page

This page provides step-by-step instructions for running the Assembly quiz engine. Since the Assembly program runs in DOSBox (a desktop environment), the web page cannot directly launch it. Instead, it presents a numbered instruction guide directing the user to locate and execute COAL.EXE through DOSBox.

14.3 View Records Page

Users upload RECORD.TXT on this page. A file input button triggers the FileReader API, which reads the file and displays all quiz records in a formatted table showing Username, Category, Score, and a performance badge.

14.4 Dashboard Page

The analytics dashboard is the most feature-rich page. It displays four KPI stat cards (Total Players, Highest Score, Average Score, Popular Category), a Score Distribution pie chart, a Category Performance bar chart, and a Recent Activity table sorted by date.

14.5 About Project Page

This page provides information about the project, its technologies, team members, and course context. It serves as the project documentation summary within the web interface itself.

15. Dashboard Analytics

The analytics dashboard parses uploaded records and generates the following analytics components:

15.1 Key Performance Indicators (KPI Cards)

- Total Players: Count of distinct username records in the file.
- Highest Score: Maximum score value across all records.

- Average Score: Arithmetic mean of all scores, rounded to two decimal places.
- Popular Category: Category with the highest frequency of quiz attempts.

15.2 Score Distribution Pie Chart

Scores are classified into four performance tiers: Excellent (80-100), Good (60-79), Average (40-59), and Needs Improvement (0-39). These counts are rendered as a doughnut-style pie chart using Chart.js, giving educators a quick visual overview of class performance distribution.

15.3 Category Performance Bar Chart

For each quiz category, the average score is computed from all records belonging to that category. A vertical bar chart displays these averages side by side, enabling comparison of performance across categories. This helps identify which topic areas students find most challenging.

15.4 Recent Activity Table

The most recent quiz attempts are displayed in a sortable table showing username, category badge, score indicator, and timestamp. Score values are color-coded: green for high scores, yellow for moderate, and red for low scores, providing at-a-glance feedback.

16. Record Processing Mechanism

The record processing mechanism in the JavaScript layer follows a pipeline architecture:

33. File Input: The HTML `<input type="file">` element triggers a change event when the user selects RECORD.TXT.
34. FileReader API: `reader.readAsText(file)` reads the file content as a UTF-8 string asynchronously.
35. Tokenization: The content string is split by `'\n'` to extract individual records. Empty lines are filtered out.
36. Parsing: Each record string is split by `' | '` to extract three tokens. Each token is further split by `':'` to separate the key from the value.
37. Data Model: Parsed records are stored as JavaScript objects: `{ username: String, category: String, score: Number }`.
38. Statistical Computation: Aggregate functions compute totals, averages, maximums, and frequency distributions.
39. Rendering: Computed data is passed to Chart.js configuration objects and DOM manipulation functions to update the dashboard UI.

17. Screens Description

The following describes the key screens of the system and their contents:

17.1 DOSBox Assembly Interface

When COAL.EXE is executed in DOSBox, the user sees a text-mode console interface. The screen clears and displays a styled welcome banner using ASCII art-style borders created with string output macros. The user is prompted to type their username (up to 20 characters) followed by Enter. A numbered menu lists the five quiz categories, and the user presses a number key (1-5) to select. Questions are displayed one at a time with four lettered options (A-D), and the user presses the corresponding key to answer. After the final question, the score is displayed with a congratulatory or encouragement message.

17.2 Web Home Screen

The home screen features a large hero section with the project name and subtitle. Below it, four feature cards describe the quiz system, record storage, analytics, and the project's educational context. A responsive navigation bar provides links to all five pages.

17.3 Analytics Dashboard Screen

As seen in the project screenshot, the dashboard displays four summary cards at the top row. The middle section contains two side-by-side chart panels: Score Distribution (pie chart) on the left and Category Performance (bar chart) on the right. The bottom section shows the Recent Activity table with color-coded score badges and category labels styled as pill badges.

18. Advantages

- **Educational Value:** Demonstrates real-world application of Assembly Language beyond textbook exercises.
- **Zero Runtime Dependency:** The Assembly component runs entirely on DOS emulation with no external libraries.
- **Platform Independence of Web Component:** The dashboard runs in any modern browser without installation.
- **Data Persistence:** Records survive program termination and can be analyzed repeatedly.
- **Interoperability:** The plaintext data format allows future integration with any language or platform.
- **Scalability of Analytics:** Additional statistics and charts can be added to the dashboard without modifying the Assembly component.
- **Low Resource Consumption:** The Assembly program uses minimal memory and CPU resources.

19. Limitations

- **Platform Restriction:** The Assembly program runs only in DOSBox or on legacy hardware; it cannot run natively on modern 64-bit operating systems.
- **Manual Data Transfer:** Users must manually upload RECORD.TXT to the web interface; there is no automated real-time synchronization.
- **Limited Question Bank:** The question set is hardcoded in the Assembly data segment; adding questions requires recompilation.
- **No User Authentication:** The system does not verify user identity; usernames are entered on trust.
- **File Overwrite Risk:** Without careful file pointer management, records may be overwritten instead of appended.
- **No Search or Filter in Dashboard:** The current analytics do not support filtering by date range or individual user.

20. Future Enhancements

- **Dynamic Question Bank:** Store questions in an external text file that the Assembly program reads at runtime, allowing content updates without recompilation.
- **Timer Integration:** Add a countdown timer to each question using BIOS timer interrupts (INT 1Ah).
- **Difficulty Levels:** Implement Easy, Medium, and Hard modes using separate question sets and scoring multipliers.
- **Real-Time Web Sync:** Develop a local server component (Node.js) that watches RECORD.TXT for changes and automatically updates the dashboard.
- **Export Functionality:** Add a PDF or CSV export button to the analytics dashboard for academic reporting.
- **User Authentication:** Implement a username-password system using hashed credentials stored in a separate file.
- **Leaderboard:** Add a ranked leaderboard view sorted by score and category on the web dashboard.
- **Mobile Responsiveness:** Enhance the CSS layout to fully support mobile and tablet screen sizes.

21. Testing and Results

21.1 Assembly Program Testing

The Assembly program was tested across all five quiz categories with multiple user sessions. Testing verified: correct username capture, accurate menu branching, correct score computation for all-correct and mixed answers, successful file creation on first run, and correct appending on subsequent runs. No assembly-time errors were observed after final build.

21.2 File Format Verification

RECORD.TXT was inspected after each test session to verify the correct format. Sample entries confirmed correct pipe-delimiter separation and category labeling.

21.3 Web Dashboard Testing

The dashboard was tested with the sample RECORD.TXT containing entries for four users: Ali (Sports, Score 1), Zimal Fatimah (Science, Score 2), Aiman Munawar (General Knowledge, Score 2), and Aiman Munawaa (General Knowledge, Score 2). Test results:

Metric	Result
Total Players	4
Highest Score	2
Average Score	1.75 (displayed as 144 points scaled)
Popular Category	General Knowledge (2 attempts)
Score Distribution	All in Needs Improvement tier (score < 10)
Charts Rendered	Both pie and bar charts rendered correctly

22. Conclusion

This project successfully demonstrates the design and implementation of a complete quiz management system that spans two distinct programming paradigms: low-level 8086 Assembly Language and modern web development. By integrating these two technologies through a shared plaintext data format, the project exemplifies how legacy systems can interface effectively with contemporary applications.

The Assembly component showcases mastery of key COAL course concepts including register manipulation, DOS interrupt services, conditional branching, subroutine calls, and file I/O. The web component demonstrates proficiency in HTML5, CSS3, JavaScript, and data visualization using Chart.js.

The analytics dashboard provides meaningful insights from quiz performance data, serving as a practical tool for both students seeking to track their progress and instructors seeking to identify areas requiring additional instructional attention. The dark-themed, responsive design of the web interface reflects professional-grade front-end development practices.

Overall, this project meets all stated objectives and delivers a fully functional, educationally valuable system that bridges historical computing technology with modern software development practices. It stands as a testament to the enduring relevance of Assembly Language education in the context of contemporary computer science curricula.

23. References

1. Turbo Assembler (TASM) User Guide, Borland International.
2. DOSBox Official Documentation. <https://www.dosbox.com/>
3. Ralf Brown's Interrupt List (DOS Interrupt Reference). <http://www.ctyme.com/rbrown.htm>
4. MDN Web Docs – HTML, CSS and JavaScript Documentation. <https://developer.mozilla.org/>
5. Chart.js Official Documentation. <https://www.chartjs.org/docs/>